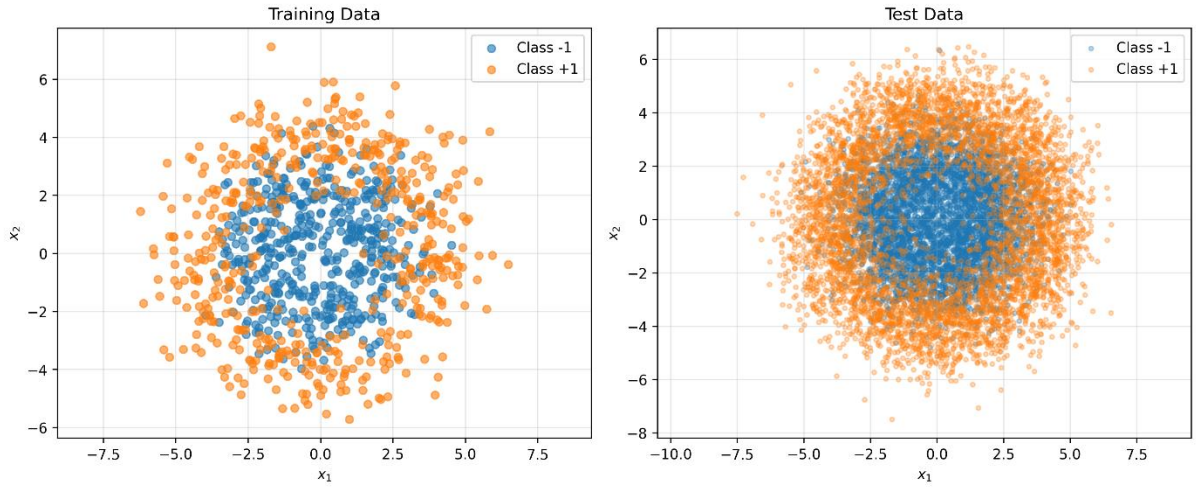
Github Code Link

Q1

Data was generated for training and testing as mentioned in the question, which looked like two concentric circles, and also, due to noise added to the data, it was not a smooth circle. Class with the label **-1** had radius 2 while class **1** had radius of 4. Noise was defined as AWGN$\sim N(0,1)$. All samples in the training and testing datasets are iid in nature. Training has 1000 samples, and testing has 10000 samples. The image below shows the visualization of the training and testing datasets



First, SVM was tested with varying parameters for kernel width, box constraint, and k-fold. The **rbf** (radial basis function) kernel was used as the boundary produced for this dataset would be non-linear in nature

gamma_values (kernel width) = [0.01, 0.05, 0.1, 0.5, 1.0, 2.0, 5.0]

C_values (box constraint) = [0.1, 0.5, 1.0, 5.0, 10.0, 50.0, 100.0]

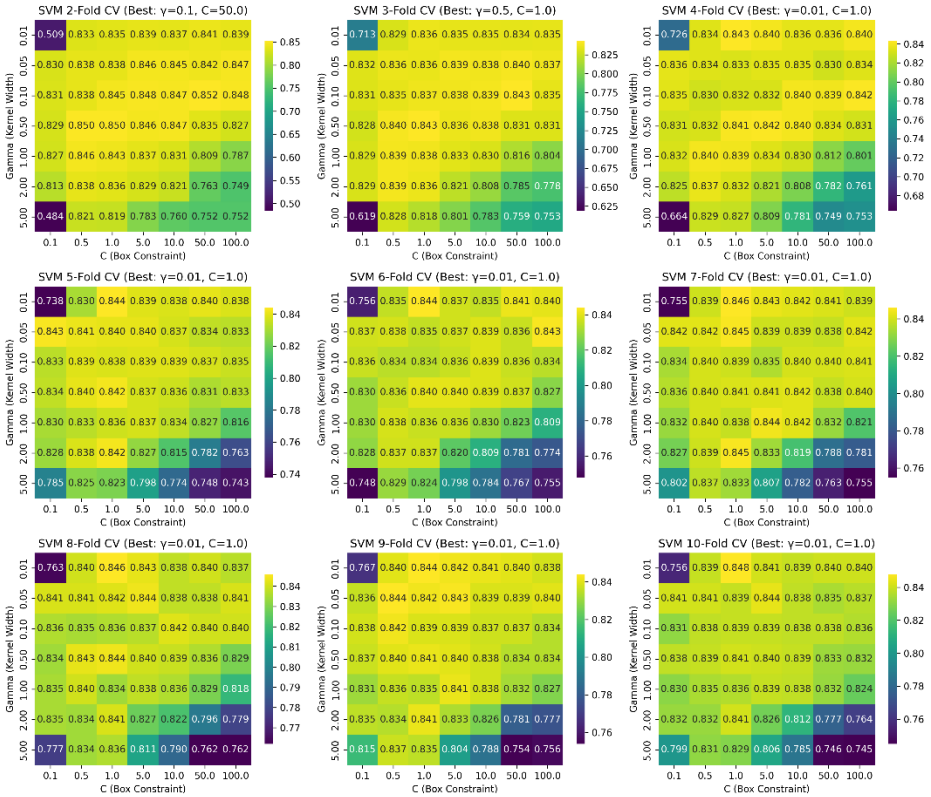
k_folds_range = [2,3,4,5,6,7,8,9,10]

The combination that had the highest log-likelihood value was used as the final model to be tested on the testing dataset. The table below summarizes the final results

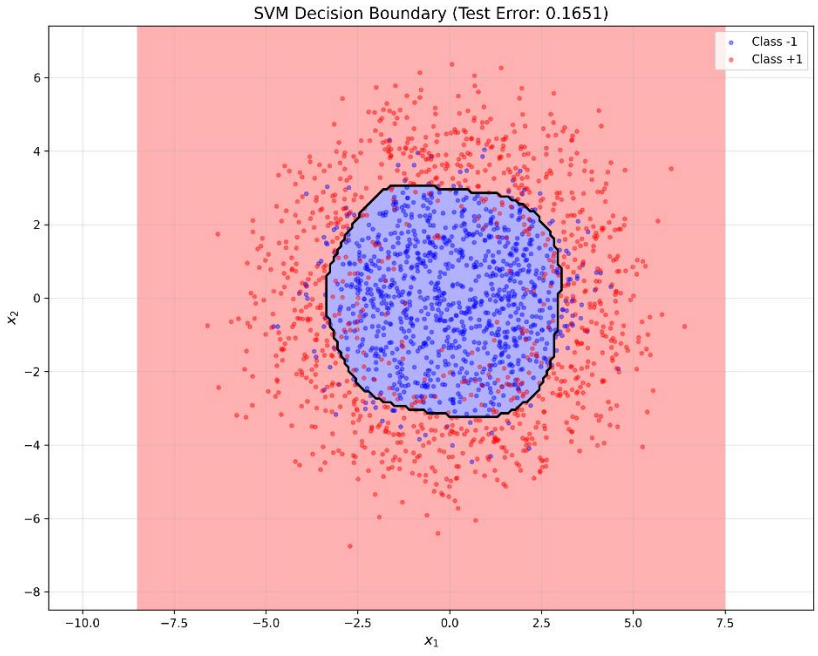
2-fold	gamma=0.1	C=50.0	acc=0.852
3-fold	gamma=0.5	C=1.0	acc=0.843
4-fold	gamma=0.01	C=1.0	acc=0.843
5-fold	gamma=0.01	C=1.0	acc=0.844
6-fold	gamma=0.01	C=1.0	acc=0.844
7-fold	gamma=0.01	C=1.0	acc=0.846
8-fold	gamma=0.01	C=1.0	acc=0.846
9-fold	gamma=0.01	C=1.0	acc=0.8441
10-fold	gamma=0.01	C=1.0	acc=0.848

We can see from the table above that the first entry of the combination was selected

Image below summarizes the different log-likelihood values obtained for all combinations of kernel width, box constraint, and k values.



The image below is the final SVM model performance on the testing dataset with prob_error = 0.1651

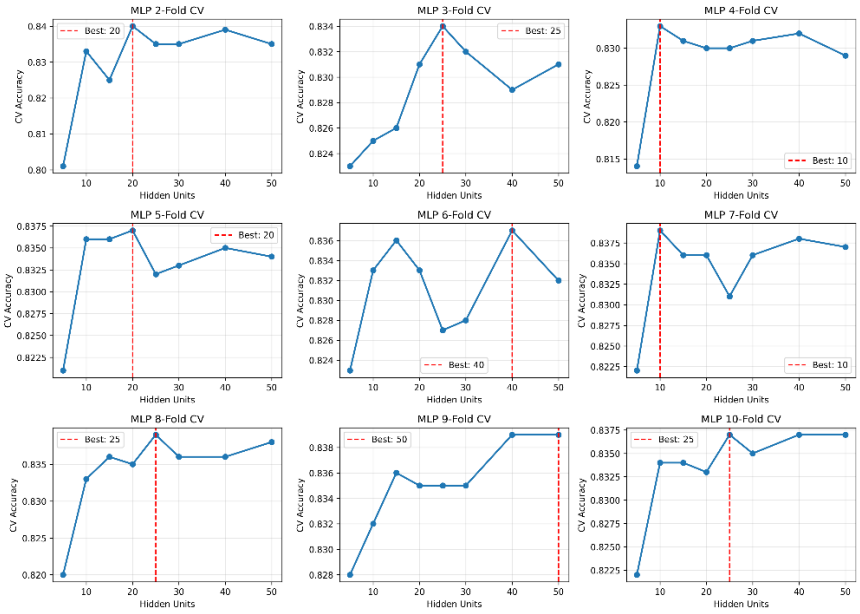


In the case of MLP, a 2 fully connected network was used with the number of perceptrons being varied from [5,10,15,20,25,30,40,50]. The hidden layer used ‘tanh’ as activation function, and the output layer used ‘softmax’ to convert outputs from each perceptron into a probability. A similar

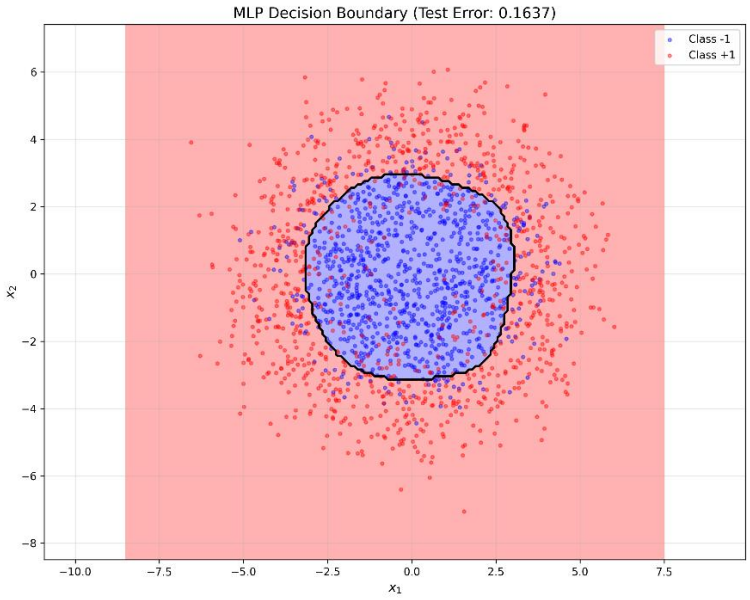
approach to SVM was used, by taking the combination with the highest validation value was chosen as the final model. The table below summarizes the values obtained for all combinations

2-fold	20 perceptrons	acc=0.84
3-fold	25 perceptrons	acc=0.834
4-fold	10 perceptrons	acc=0.833
5-fold	20 perceptrons	acc=0.837
6-fold	40 perceptrons	acc=0.837
7-fold	10 perceptrons	acc=0.839
8-fold	25 perceptrons	acc=0.839
9-fold	50 perceptrons	acc=0.839

The final model was chosen as a hidden layer having 20 perceptrons with 2-fold cross-validation. The image below summarizes the performance of MLP with different perceptrons in hidden layer.

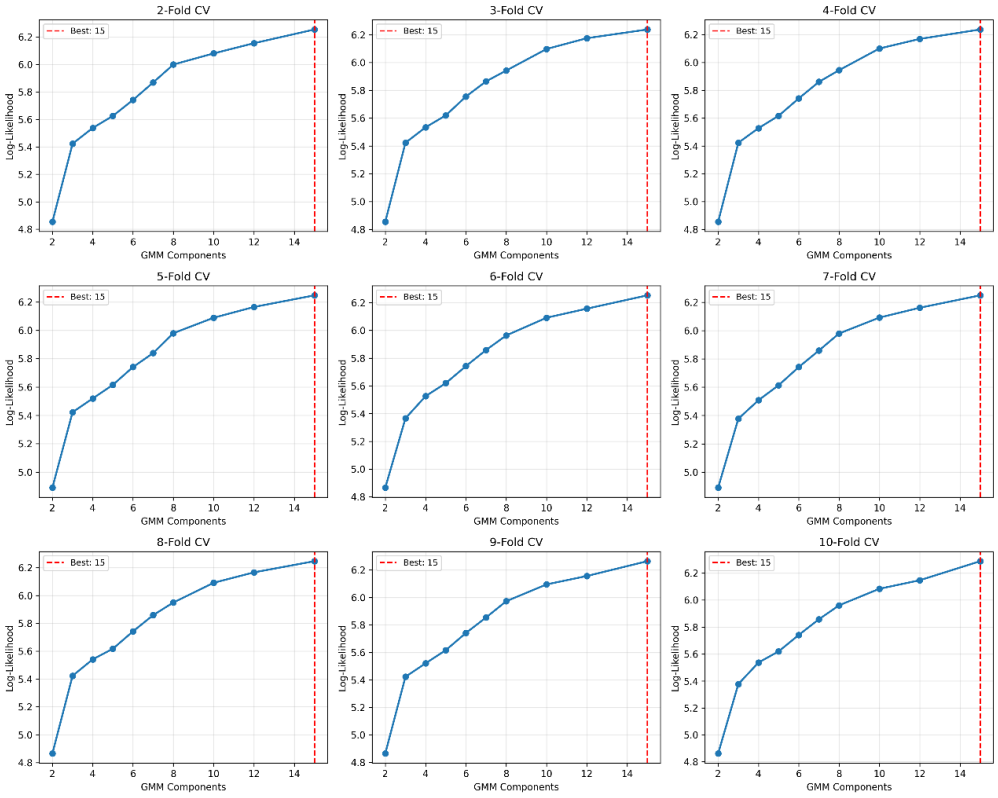


The output below is the final model of MLP on the testing dataset with prob_error = 0.1637



Q2

First, I downloaded this Zebra colored image locally. For preprocessing, a 5-dimensional feature vector was generated for each pixel by appending row index, column index, red value, green value, and blue value for each pixel into a raw feature vector and normalizing the values in the range of [0,1] for each feature. To fit a GMM to segment this image, I looped from 2 to 15 Gaussian components. For each component, maximizing log-likelihood was used as the objective function using 2-10 fold cross-validation. The Gaussian component with the highest k-fold cross-validation was chosen as the final model. The image below shows different Gaussian components with their respective log-likelihood for a specific k-fold validation.



The final model achieved a log-likelihood score of 6.2604 with 15 segmentations and 10-fold cross-validation. The image below shows a comparison between the original image and the segmented image.

The table below summarizes different segments and the number of pixels belonging to each label

Segment #	# of pixels
Segment 0	17443 pixels (11.30%)
Segment 1	4297 pixels (2.78%)
Segment 2	5967 pixels (3.86%)
Segment 3	3305 pixels (2.14%)
Segment 4	5160 pixels (3.34%)
Segment 5	9500 pixels (6.15%)
Segment 6	15070 pixels (9.76%)
Segment 7	9759 pixels (6.32%)
Segment 8	10085 pixels (6.53%)
Segment 9	4028 pixels (2.61%)
Segment 10	17146 pixels (11.10%)
Segment 11	13591 pixels (8.80%)
Segment 12	21652 pixels (14.02%)
Segment 13	6086 pixels (3.94%)
Segment 14	11312 pixels (7.33%)

References

https://en.wikipedia.org/wiki/Radial_basis_function_kernel

https://docs.pytorch.org/docs/stable/generated/torch.nn.Softmax.html